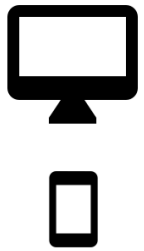
GraphQL

Modern API Query Language

What is GraphQL?

- GraphQL is a **query language for APIs** and a runtime for fulfilling those queries with your existing data
- Provides a complete and understandable description of the data in your API
- Gives clients the power to ask for **exactly what they need** and nothing more
- Makes it easier to evolve APIs over time
- Enables powerful developer tools

1



HTTP GET

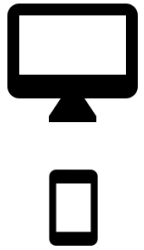
```
{
  “user”: {
    “id”: “er3tg439frjw”
    “name”: “Mary”,
    “address”: { … },
    “birthday”: “July 26, 1982”
  }
}
```

/users/<id>

/users/<id>/posts

/users/<id>/followers

2



HTTP GET

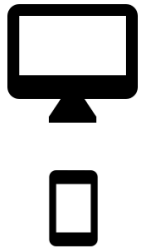
```
{
  “posts”: [{
    “id”: “ncwon3ce89hs”
    “title”: “Learn GraphQL today”,
    “content”: “Lorem ipsum … ”,
    “comments”: [ … ],
  }]
}
```

/users/<id>

/users/<id>/posts

/users/<id>/followers

3



```
{
  “followers”: [{
    “id”: “leo83h2dojsu”
    “name”: “John”,
    “address”: { … },
    “birthday”: “July 26, 1982”
  },
  …]
}
```

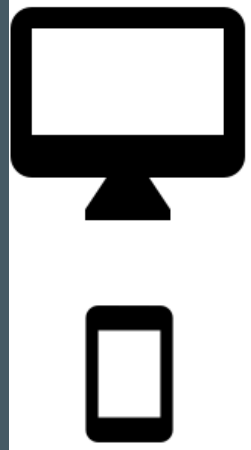
HTTP GET

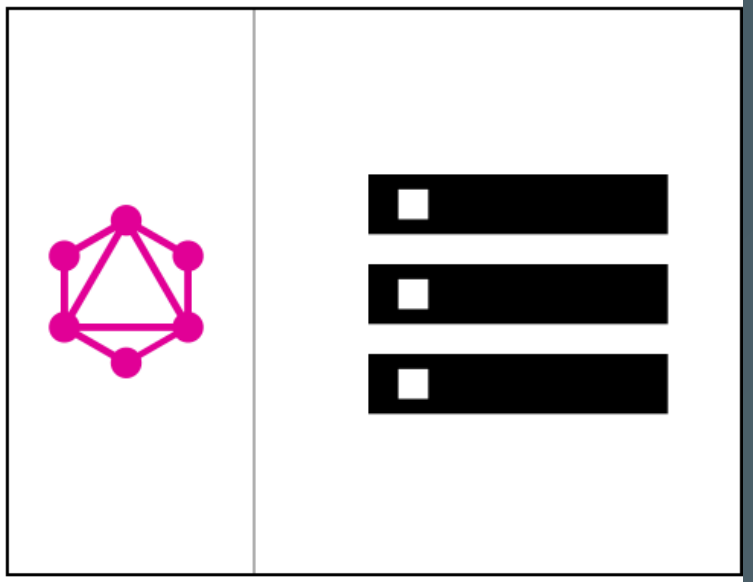
/users/<id>

/users/<id>/posts

/users/<id>/followers

```
query {
  User(id: “er3tg439frjw”) {
    name
    posts {
      title
    }
    followers(last: 3) {
      name
    }
  }
}
```





```
{
  “data”: {
    “User”: {
      “name”: “Mary”,
      “posts”: [
        { title: “Learn GraphQL today” }
      ],
      “followers”: [
        { name: “John” },
        { name: “Alice” },
        { name: “Sarah” },
      ]
    }
  }
}
```

GraphQL History

- **2012**: GraphQL developed internally by Facebook for mobile apps
- **2015**: Facebook publicly released GraphQL as an open specification
- **2018**: GraphQL moved to the GraphQL Foundation (Linux Foundation)
- **Today**: Used by GitHub, Shopify, Twitter, Airbnb, and thousands of companies

GraphQL vs REST

REST	GraphQL
Multiple endpoints	Single endpoint
Fixed data structure	Flexible data structure
Over-fetching/Under-fetching	Ask for exactly what you need
Version-based (v1, v2)	Versionless evolution

Note: GraphQL is a query language for APIs, **not databases**!

Common Problems with REST

```
// REST: Multiple requests needed
GET /users/1            → { id: 1, name: "Alice" }
GET /users/1/posts      → [ { id: 1, title: "..." }, ... ]
GET /posts/1/comments   → [ { id: 1, text: "..." }, ... ]

// GraphQL: One request, exact data
query {
  user(id: 1) {
    name
    posts {
      title
      comments {
        text
      }
    }
  }
}
```

Why GraphQL?

✅ **No Over-fetching**: Get only the fields you need
✅ **No Under-fetching**: Get related data in one request
✅ **Strongly Typed**: Schema defines what's possible
✅ **Introspection**: Clients can discover API capabilities
✅ **Developer Experience**: Excellent tooling (Apollo Studio, GraphiQL)
✅ **Evolutionary**: Add fields without breaking existing queries

Core Concepts: Query

Queries are for **reading** data (like GET in REST)

```
# Get all users
query {
  users {
    id
    username
    email
  }
}
```

```
# Get specific user with posts
query {
  user(id: "1") {
    username
    posts {
      id
      title
    }
  }
}
```

Core Concepts: Mutation

Mutations are for **modifying** data (like POST/PUT/DELETE in REST)

```
mutation {
  createUser(input: {
    username: "alice"
    email: "alice@example.com"
    password: "secret123"
  }) {
    id
    username
    email
    createdAt
  }
}
```

Core Concepts: Subscription

Subscriptions are for **real-time** updates via WebSocket

```
subscription {
  newPost {
    id
    title
    author {
      username
    }
  }
}
```

When a new post is created, all subscribed clients receive the update!

Type System

GraphQL has a **strongly-typed schema** that defines your API

```
type User {
  id: ID!                # ! means required
  username: String!
  email: String!
  posts: [Post!]!        # Array of Post objects
  createdAt: DateTime
}

type Post {
  id: ID!
  title: String!
  content: String!
  author: User!          # Relationship to User
}
```

Scalar Types

Built-in scalars:

- `String` - UTF-8 text
- `Int` - 32-bit integer
- `Float` - Signed double-precision floating-point
- `Boolean` - true or false
- `ID` - Unique identifier (serialized as String)

Scalar Types

Custom scalars:

- `DateTime` - ISO 8601 date-time
- `Email` - Email address validation
- `URL` - Valid URLs

Object Types and Fields

```
type User {
  # Scalar fields
  id: ID!
  username: String!

  # Object field (relationship)
  profile: UserProfile

  # List field
  posts: [Post!]!

  # Field with arguments
  posts(status: PostStatus, limit: Int): [Post!]!
}
```

Input Types

Used for complex arguments in mutations

```
input CreateUserInput {
  username: String!
  email: String!
  password: String!
}

mutation {
  createUser(input: {
    username: "bob"
    email: "bob@example.com"
    password: "pass123"
  }) {
    id
    username
```

Defining a Schema: Query

The **Query** type defines all read operations

```
type Query {
  # Get all users
  users(limit: Int = 10, offset: Int = 0): [User!]!

  # Get single user by ID
  user(id: ID!): User

  # Search
  search(query: String!): [SearchResult!]!

  # Current authenticated user
  me: User
}
```

Defining a Schema: Mutation

The **Mutation** type defines all write operations

```
type Mutation {
  # Authentication
  register(input: CreateUserInput!): AuthResult!
  login(username: String!, password: String!): AuthResult!

  # Post operations
  createPost(input: CreatePostInput!): Post!
  updatePost(id: ID!, input: UpdatePostInput!): Post!
  deletePost(id: ID!): Boolean!
}
```

Defining a Schema: Subscription

The **Subscription** type defines real-time events

```
type Subscription {
  # Listen for new posts
  postCreated: Post!

  # Listen for updates to a specific post
  postUpdated(id: ID!): Post!

  # Listen for new comments on a post
  commentAdded(postId: ID!): Comment!
}
```

Resolvers: The 4 Parameters

Resolvers are functions that return values for fields

```
const resolvers = {
  Query: {
    user: (parent, args, context, info) => {
      // parent: result from parent resolver
      // args: { id: "1" } - field arguments
      // context: shared data (user, db, pubsub)
      // info: query metadata (rarely used)

      return context.db.getUserById(args.id);
    }
  }
};
```

Resolver Example

```
const resolvers = {
  Query: {
    users: (_, { limit, offset }, context) => {
      return context.db.getUsers(limit, offset);
    },
    user: (_, { id }, context) => {
      return context.db.getUser(id);
    }
  },

  User: {
    // Nested resolver for User.posts field
    posts: (parent, _, context) => {
      return context.db.getPostsByAuthor(parent.id);
    }
  }
};
```

Context: Sharing Data

Context is created per-request and shared across all resolvers

```
const createContext = async ({ req }) => {
  // Extract auth token
  const token = req.headers.authorization?.replace('Bearer ', '');

  // Verify and get current user
  const currentUser = token ? await verifyToken(token) : null;

  return {
    currentUser,  // Authenticated user
    db: database, // Database connection
    pubsub,       // For subscriptions
  };
};
```

Advanced: Fragments

Fragments are reusable field selections

```
fragment UserFields on User {
  id
  username
  email
}

query {
  alice: user(id: "1") {
    ...UserFields
  }
  bob: user(id: "2") {
    ...UserFields
  }
}
```

Advanced: Interfaces

Interfaces define shared fields across types

```
interface Node {
  id: ID!
  createdAt: DateTime!
}

type User implements Node {
  id: ID!
  createdAt: DateTime!
  username: String!
}
```

Advanced: Union Types

Unions represent multiple possible return types

```
union SearchResult = User | Post | Comment

query {
  search(query: "graphql") {
    __typename  # Tells which type
    ... on User {
      username
    }
    ... on Post {
      title
    }
    ... on Comment {
      text
    }
```

Error Handling

GraphQL returns **both** data and errors

```
{
  "data": {
    "user": null
  },
  "errors": [
    {
      "message": "User not found",
      "extensions": {
        "code": "NOT_FOUND"
      },
      "path": ["user"]
    }
  ]
}
```

Authentication & Authorization

```
const resolvers = {
  Query: {
    me: (_, __, context) => {
      if (!context.currentUser) {
        throw new GraphQLError('Not authenticated', {
          extensions: { code: 'UNAUTHENTICATED' }
        });
      }
      return context.currentUser;
    }
  },

  Mutation: {
    createPost: (_, { input }, context) => {
      // Require authentication
      if (!context.currentUser) {
        throw new GraphQLError('Not authenticated');
      }
      return createPost(input, context.currentUser);
    }
  }
};
```

GraphQL Server

Popular server implementations:

- **Apollo Server** - Full-featured, production-ready
  - `npm install @apollo/server graphql`
  - Supports subscriptions, plugins, caching
- **GraphQL.js** - Reference implementation
  - Lower-level, more control
- **GraphQL Yoga** - Lightweight alternative

Apollo Server Setup

```
import { ApolloServer } from '@apollo/server';
import { startStandaloneServer } from '@apollo/server/standalone';

const server = new ApolloServer({
  typeDefs,  // Your schema
  resolvers, // Your resolvers
});

const { url } = await startStandaloneServer(server, {
  listen: { port: 4000 },
  context: async ({ req }) => ({
    currentUser: await getUserFromToken(req.headers.authorization)
  })
});

console.log(`Server ready at ${url}`);
```

GraphQL Client

Popular client libraries:

- **Apollo Client** - Most popular, feature-rich
  - Intelligent caching
  - React integration
  - Subscriptions support
- **Relay** - Facebook's client, optimized for performance
- **URQL** - Lightweight alternative
- **graphql-request** - Minimal client

Apollo Client Setup

```
import { ApolloClient, InMemoryCache, HttpLink } from '@apollo/client';

const client = new ApolloClient({
  link: new HttpLink({
    uri: 'http://localhost:4000/graphql',
  }),
  cache: new InMemoryCache(),
});
```

Apollo Client: useQuery

```
import { useQuery, gql } from '@apollo/client';

const GET_USERS = gql`
  query GetUsers {
    users {
      id
      username
    }
  }
`;

function UserList() {
  const { loading, error, data } = useQuery(GET_USERS);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;

  return (
    <ul>
      {data.users.map(user => (
        <li key={user.id}>{user.username}</li>
      ))}
    </ul>
  );
}
```

Apollo Client: useMutation

```
const CREATE_POST = gql`
  mutation CreatePost($input: CreatePostInput!) {
    createPost(input: $input) {
      id
      title
    }
  }
`;

function CreatePost() {
  const [createPost, { loading }] = useMutation(CREATE_POST);

  const handleSubmit = async (e) => {
    e.preventDefault();
    await createPost({
      variables: {
        input: { title, content }
      }
    });
  };

  return <form onSubmit={handleSubmit}>...</form>;
}
```

Apollo Client: useSubscription

```
const POST_CREATED = gql`
  subscription OnPostCreated {
    postCreated {
      id
      title
    }
  }
`;

function PostFeed() {
  const { data } = useSubscription(POST_CREATED, {
    onData: ({ data }) => {
      console.log('New post:', data);
    }
  });

  return <div>Latest: {data?.postCreated.title}</div>;
}
```

Tools & Resources

Development Tools:

- Apollo Studio - GraphQL IDE and monitoring
- GraphiQL - In-browser IDE
- Postman - Supports GraphQL queries

Learning:

- GraphQL.org - Official docs
- How to GraphQL - Tutorials
- Apollo Docs - Comprehensive guides

Sample Code

📁 **Complete demo available at:**

`sample_code/graphql-demo/`